FILM REVIEW SYSTEM

Dao Tien Phat - 001477385

Contents

1. Introduction 2
2. System Design and Structure 2
   2.1. Navigation Structure 2
   2.2. User Interface (UI) Design 3
   2.3. Database Schema (Entity-Relationship) 3
3. Technical Implementation & Security 4
4. Evidence of CRUD Functionality 4
   4.1. Film Entity 4
   4.2. Review Entity 7
   4.3. Author (User) Entity 11
5. Structured Testing Schedule and Input Control 13
   5.1. Input Validation & Control 13
   5.2. Testing Schedule (Test Plan) 14
6. Web Accessibility (WAI) 16
7. Discussion on Legal, Security & Business Problems 17
   7.1. Business Problems & Crowdsourcing Strategy 17
   7.2. Security Concerns & Mitigations 17
   7.3. Legal & Privacy Compliance (GDPR Context) 17
8. Conclusions and Future Recommendations 18
   8.1. Conclusions 18
   8.2. Future Recommendations 18
9. Professional Roles & Workplace Preparation Evaluation 18
10. References 19
11. AI Using Declaration 19
12. Github link 19

1. Introduction

The "Film Review System" is a comprehensive, data-driven CRUD web application built focussing on HTML5, PHP, and MySQL designed to efficiently manage User-Generated Content (UGC). By focusing on UGC, the project empowers users to actively participate, transforming a traditional, static catalog into a dynamic, community-driven platform. The system was developed on the XAMPP environment (localhost). Utilizing this local server stack ensures complete web server and database stability during the development lifecycle, allowing for rigorous testing and debugging in an isolated environment that closely mirrors real-world deployment configurations (Prichard, 2026).

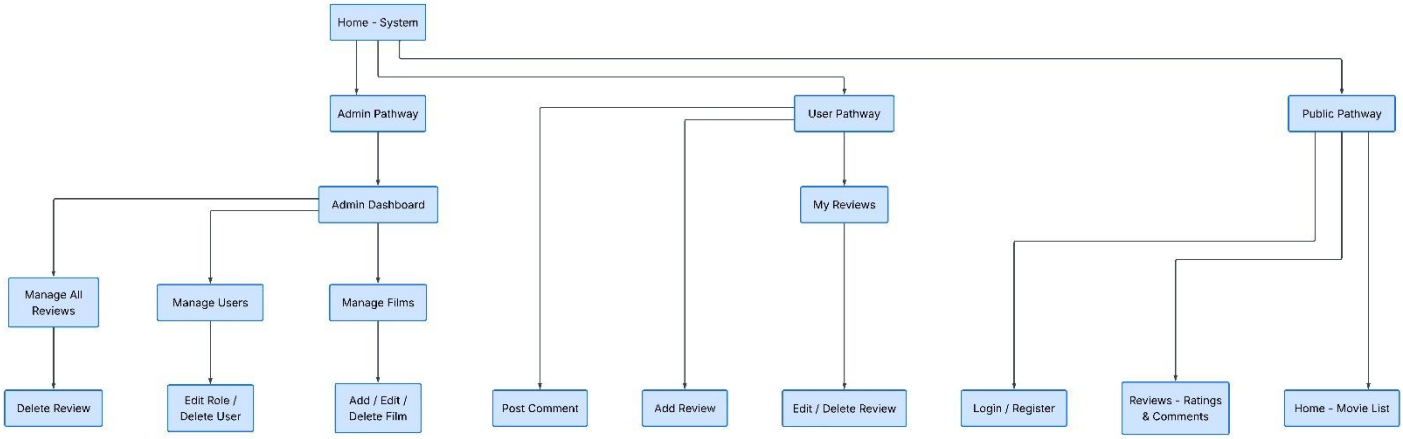
2. System Design and Structure

To ensure a seamless user experience and a robust backend logic, the system was meticulously designed focusing on intuitive navigation, modern user interface principles, and a normalized relational database.

2.1. Navigation Structure

The system's navigation is divided into two distinct pathways based on User Roles (Public/Authenticated Users vs. Administrators):

- **Public/User Pathway:** Users land on the Home page to view the film catalog. They can navigate to the Reviews section to read or join discussions. Authenticated users gain access to "My Reviews" to manage their own content, and "Add Review" to contribute new entries.
- **Admin Pathway:** Administrators possess an exclusive "Admin Dashboard" link in the primary navigation. This acts as a centralized control hub, branching out into sub-modules for managing Films, Users, and overriding system Reviews.



2.2. User Interface (UI) Design

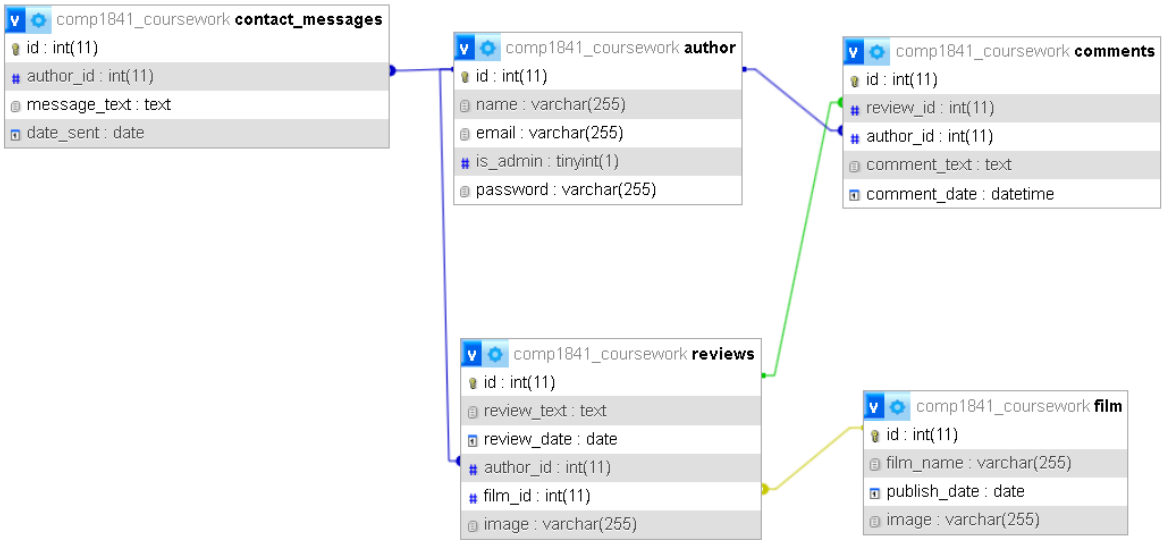
The frontend architecture abandons outdated table-based layouts in favor of modern **CSS Flexbox**. A significant design achievement is the **Master-Detail Layout** implemented on the Reviews page.

- Instead of forcing users to load a new page to view comments (which disrupts the reading flow), clicking "Comment" dynamically opens a sidebar panel (Detail) next to the review list (Master) using JavaScript DOM manipulation. This creates a fluid, Single-Page Application (SPA) feel while remaining a multi-page PHP application at its core.

2.3. Database Schema (Entity-Relationship)

The underlying MySQL database is normalized to eliminate redundancy and maintain referential integrity. The architecture consists of four primary entities:

- **Author (User):** Stores authentication details (email, hashed password, role).
- **Film:** Stores movie metadata (title, release date, poster image).
- **Review:** The central entity bridging Users and Films. It contains Foreign Keys (author_id, film_id) linking to their respective parent tables in a Many-to-One relationship.
- **Comment:** A child entity of Review, containing a Foreign Key (review_id) to establish a One-to-Many relationship between a review and its comments.
- **Referential Integrity:** ON DELETE CASCADE constraints are applied. If an Author or Film is deleted, all their dependent reviews and comments are automatically eradicated, preventing orphaned data (Prichard, 2026).



3. Technical Implementation & Security

To meet rigorous module requirements and industry standards, the system strictly utilizes **PHP Data Objects (PDO)** for all database interactions, intentionally eliminating outdated MySQLi paradigms to embrace a more secure, object-oriented approach (Prichard, 2026).

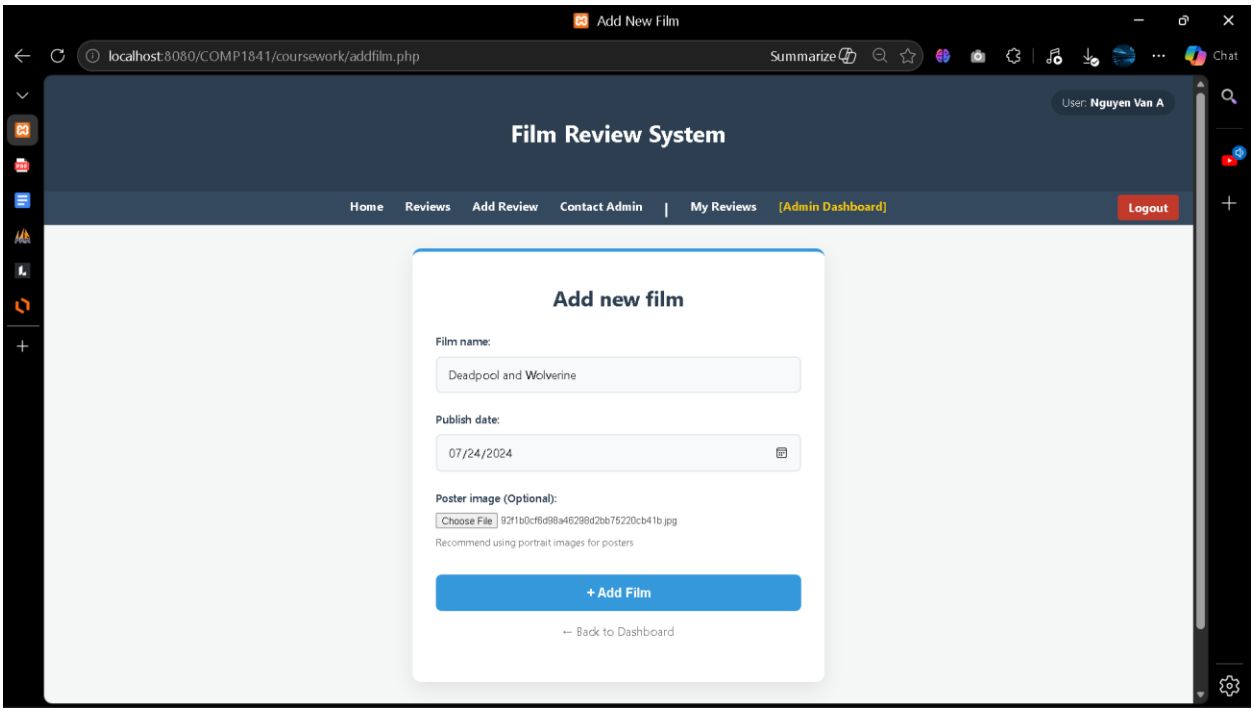
- **Preventing SQL Injection:** The implementation of the $pdo->prepare() method acts as a robust primary defense mechanism. By strictly separating the SQL query structure from the user-provided data payloads, it ensures that external inputs are treated solely as string literals. This completely neutralizes malicious query alterations, safeguarding the underlying database architecture from unauthorized data exposure or manipulation.
- **Session & Password Security:** The $_SESSION superglobal securely manages stateful authentication across stateless HTTP requests, ensuring secure access control. Furthermore, passwords are never stored in plaintext. They are encrypted using native PHP salting and hashing algorithms (such as PASSWORD_DEFAULT). The inclusion of a unique salt for each user effectively defends against sophisticated brute-force attacks and pre-computed rainbow table (matching) attacks (Prichard, 2026).

4. Evidence of CRUD Functionality

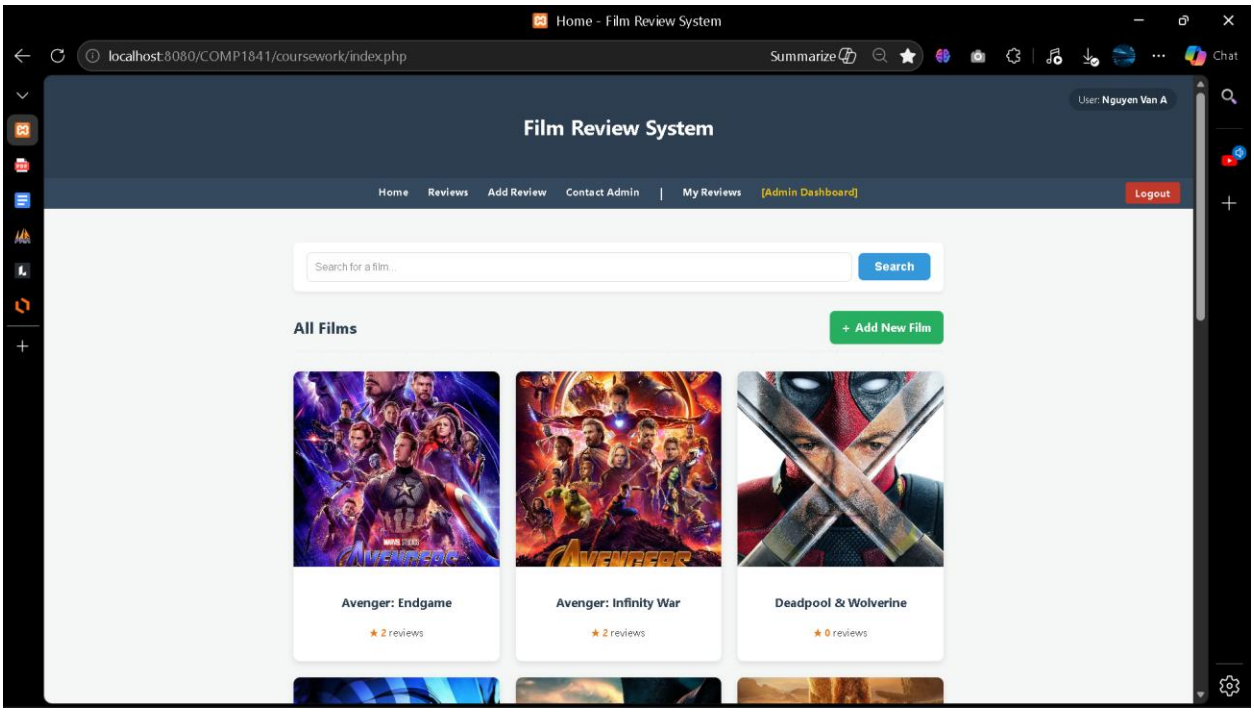
The system fully implements end-to-end CRUD operations for the Film, Review, and Author (User) entities, strictly adhering to relational database integrity.

4.1. Film Entity

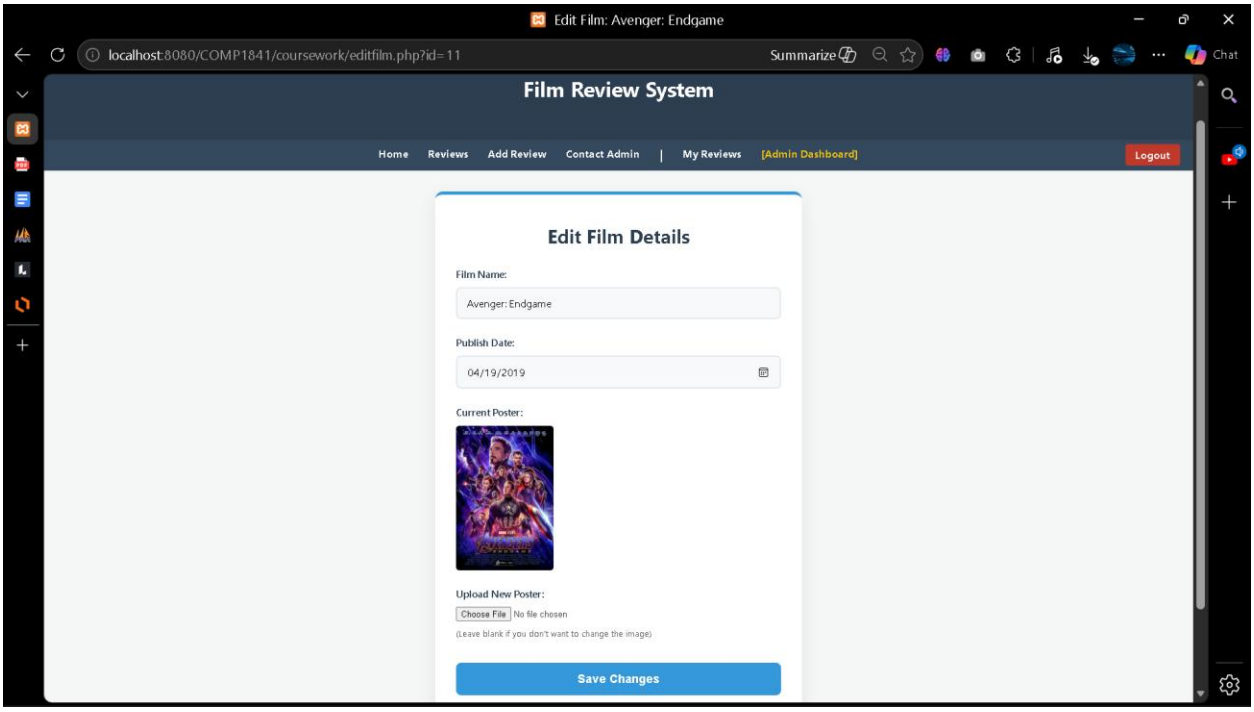
- **Create:** Administrators can intuitively add new films, requiring essential metadata such as release dates and promotional posters to maintain catalog quality.



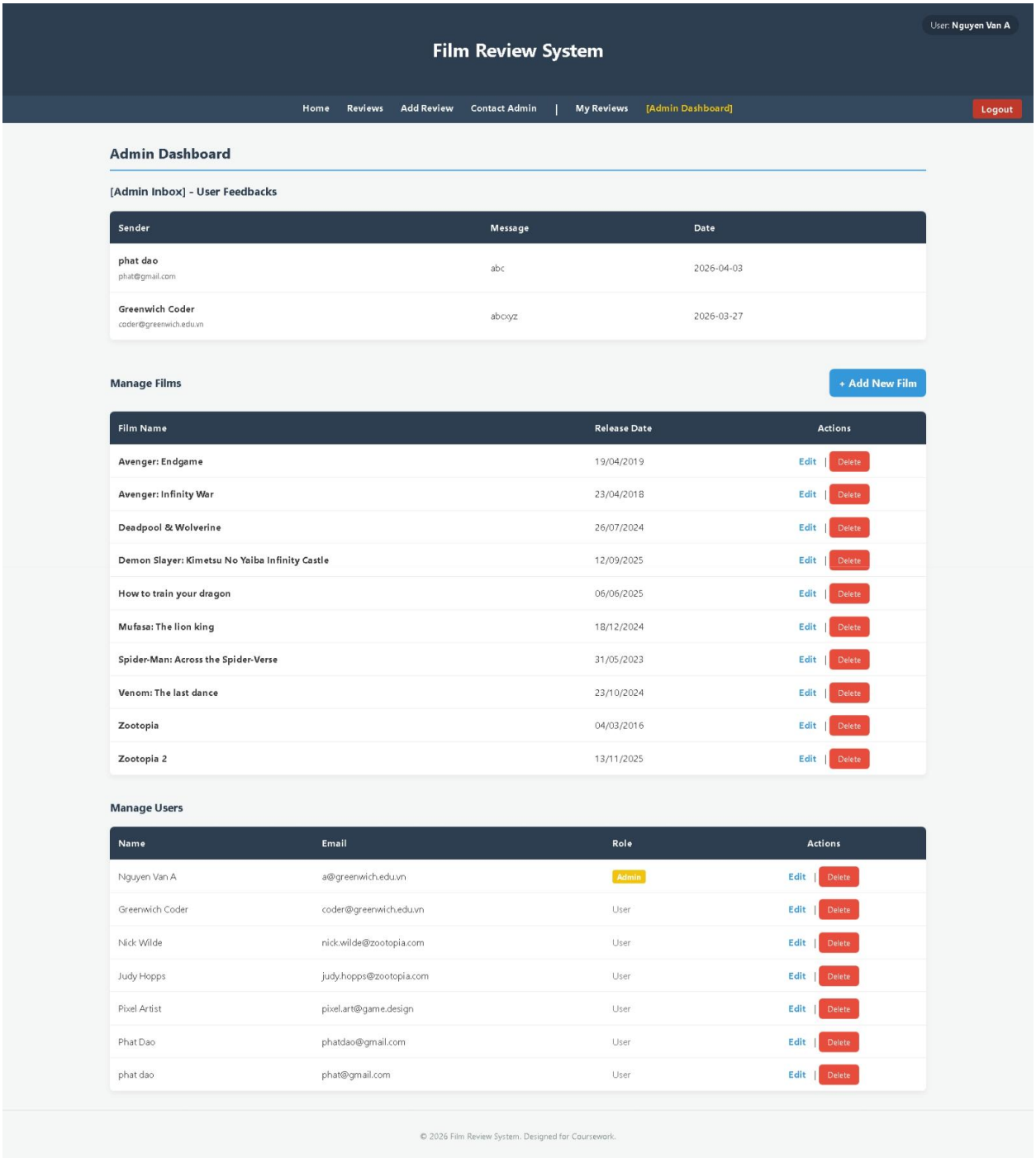
- **Read:** Films are dynamically retrieved and displayed on both the public-facing Home page grid and the secure Admin Dashboard for management oversight.



- **Update:** Administrators can modify existing film details and replace posters, ensuring the database remains accurate and up-to-date over time.

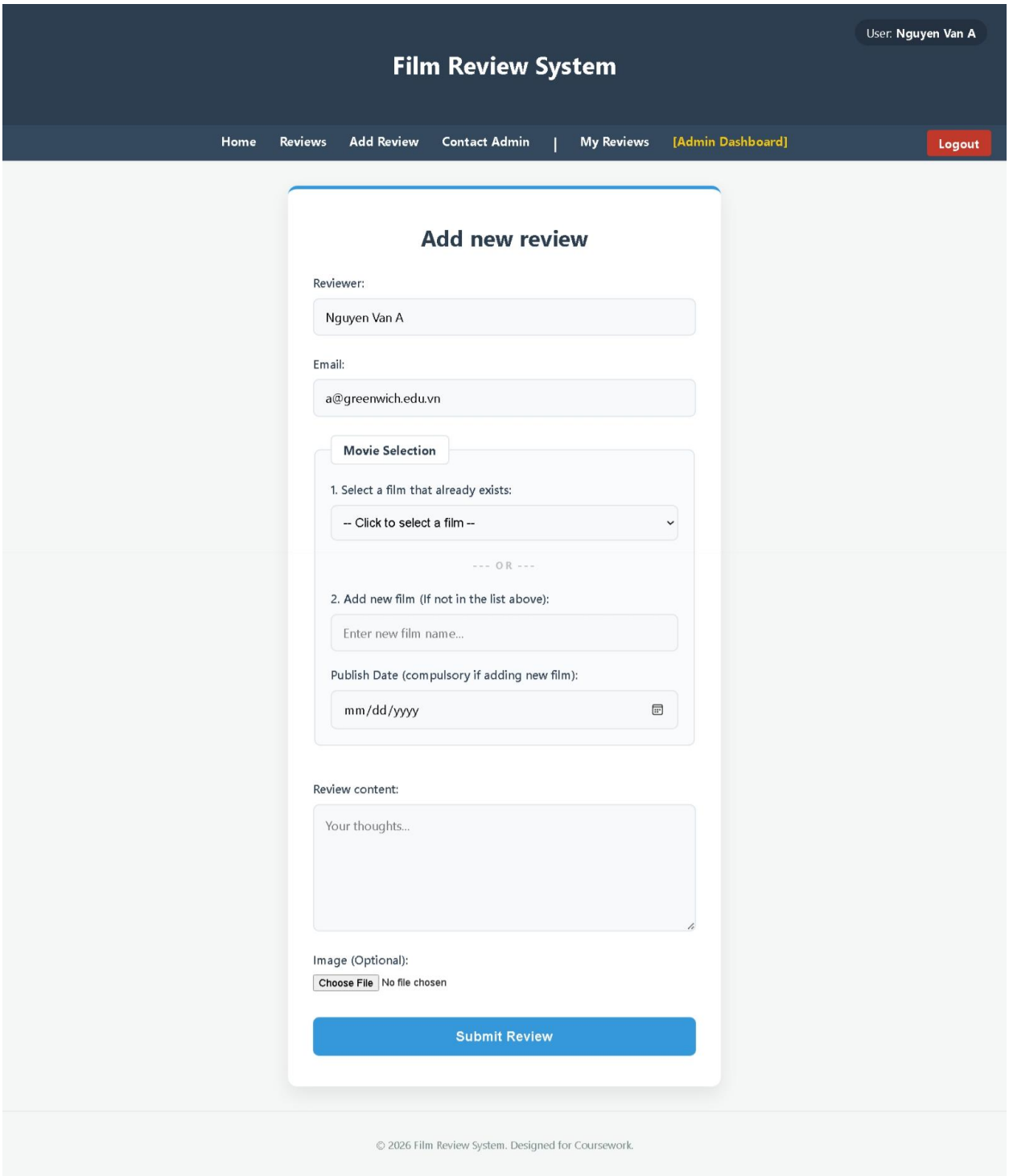


- **Delete:** Administrators can remove films from the catalog. Crucially, leveraging relational database constraints, this action cascades to automatically delete all related reviews, preventing orphaned records and maintaining referential integrity.

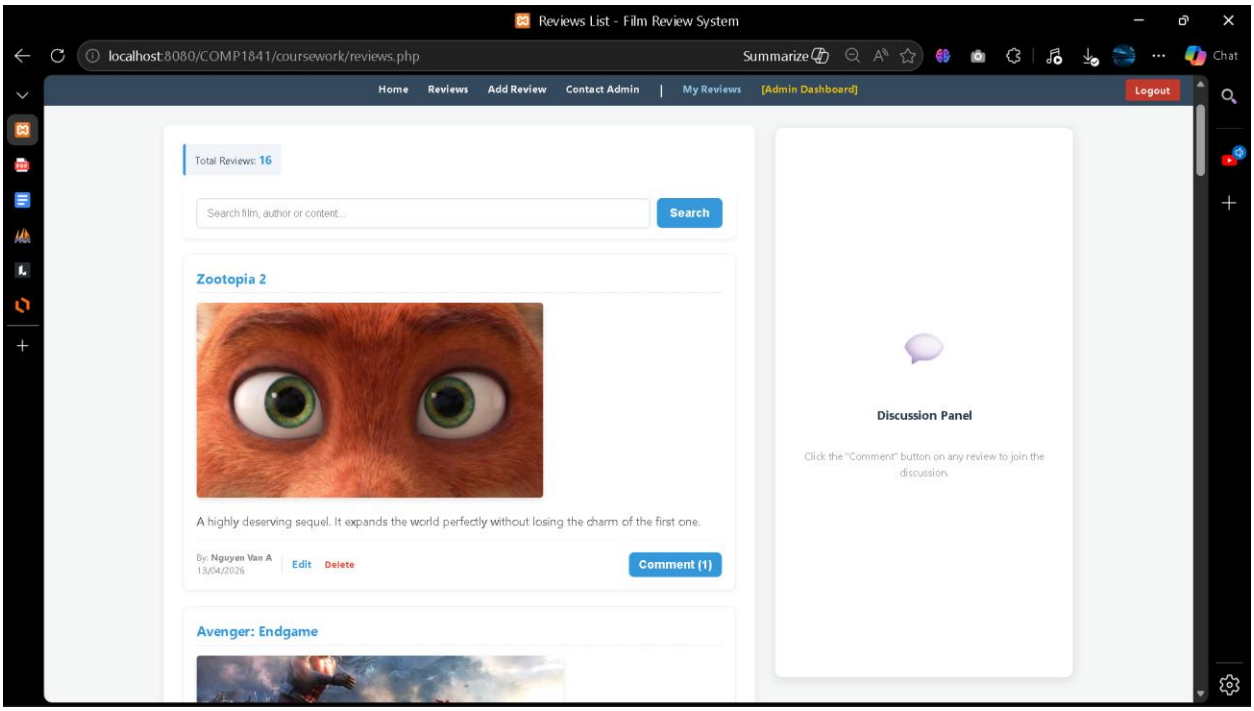


4.2. Review Entity

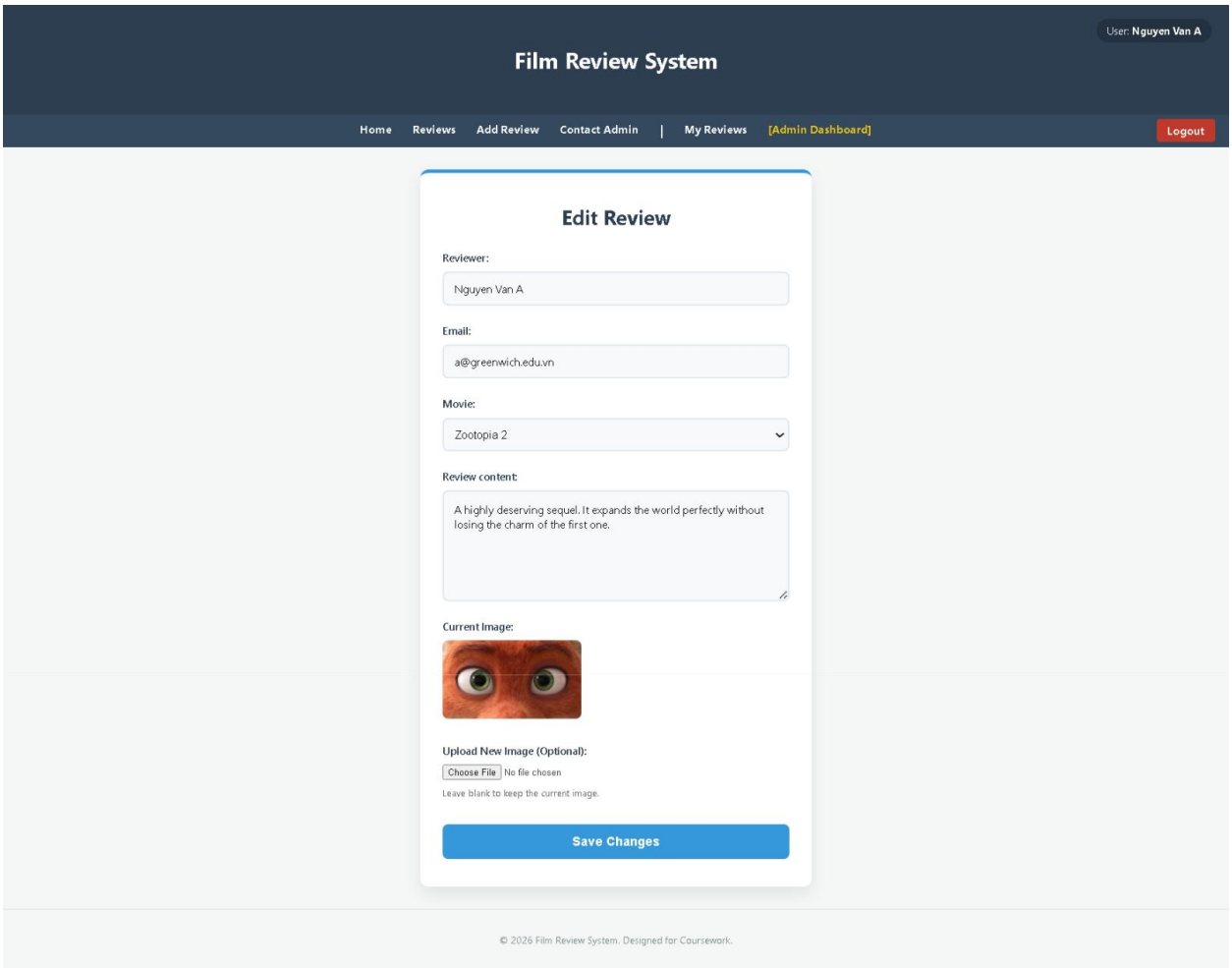
- **Create:** Authenticated users can write detailed reviews and seamlessly associate them with either pre-existing films or introduce entirely new film titles to the system.



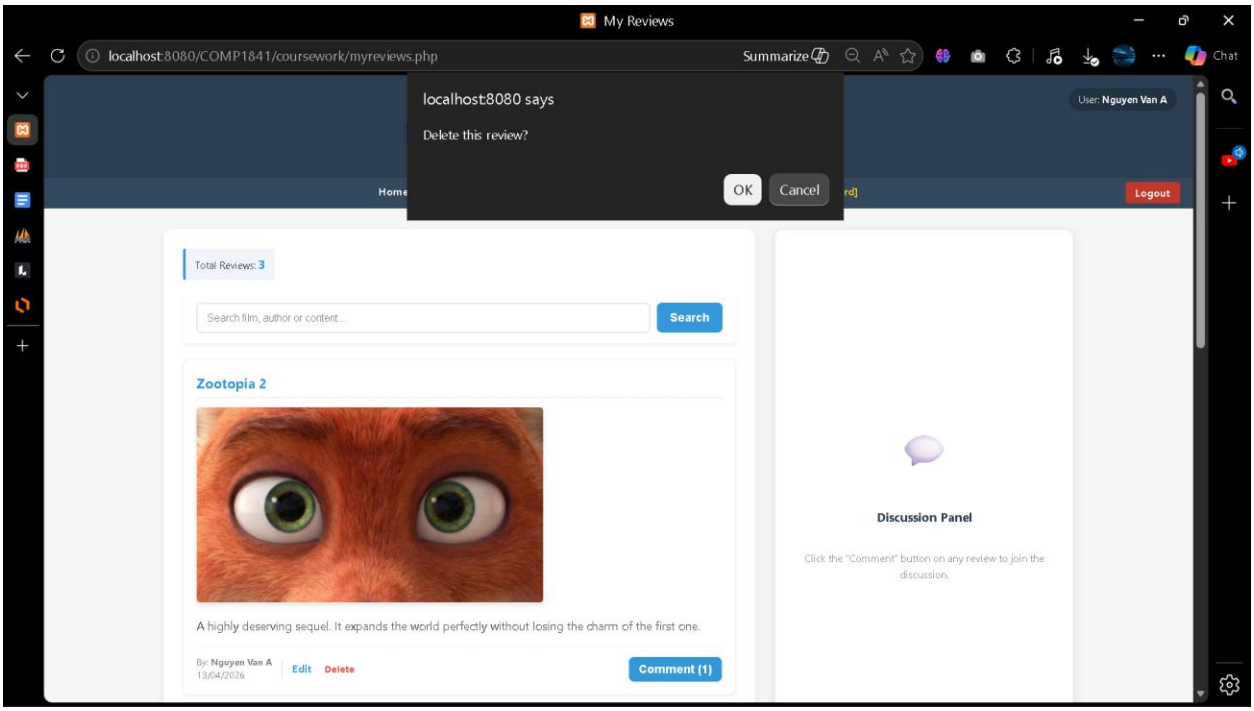
- **Read:** Reviews and their nested comments are displayed utilizing a modern Master-Detail layout, allowing users to browse content without losing their navigational context.



- **Update:** Authors retain full ownership of their content, with the ability to edit their own review text and update supplementary images as their opinions evolve.

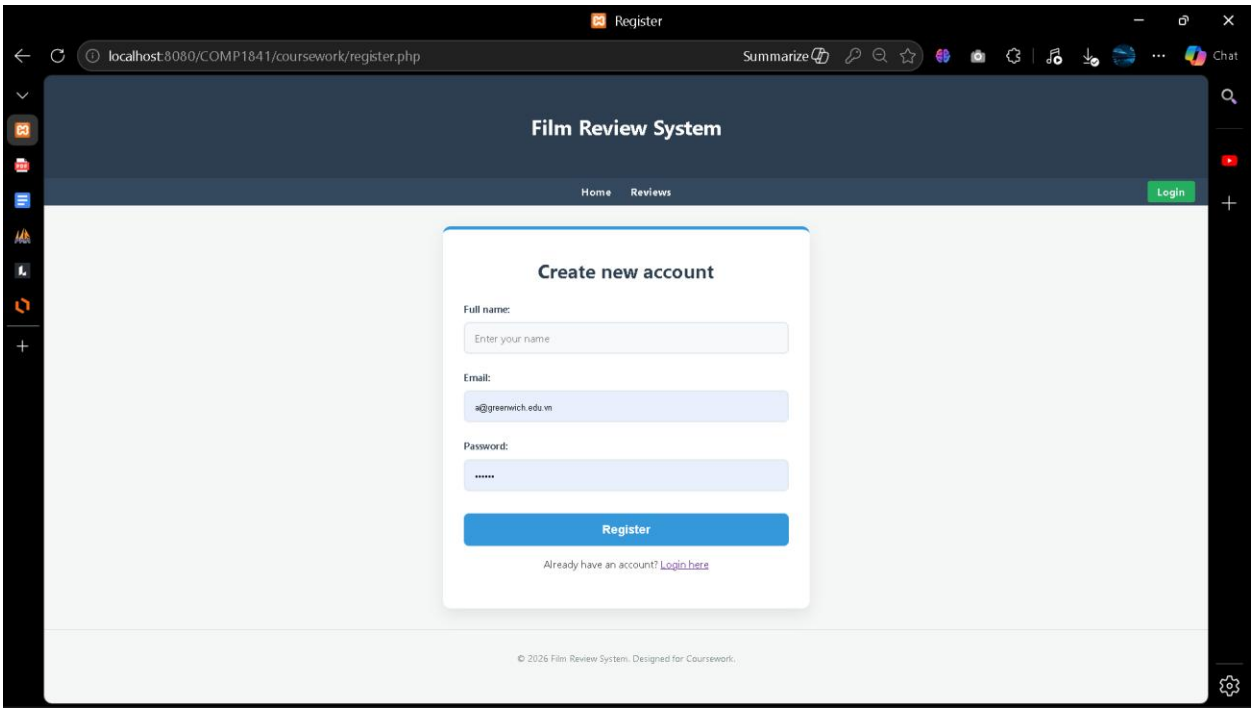


- **Delete:** Authors have the autonomy to delete their own reviews, while Administrators retain global deletion rights to moderate the platform effectively.

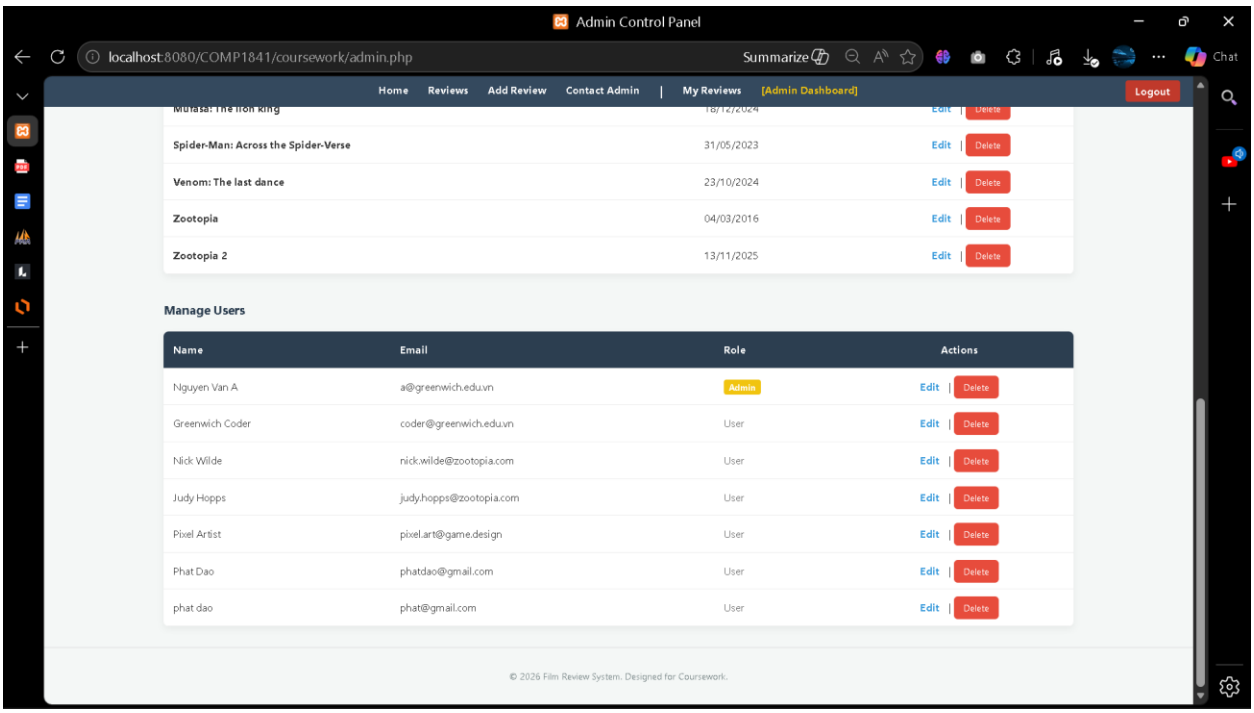


4.3. Author (User) Entity

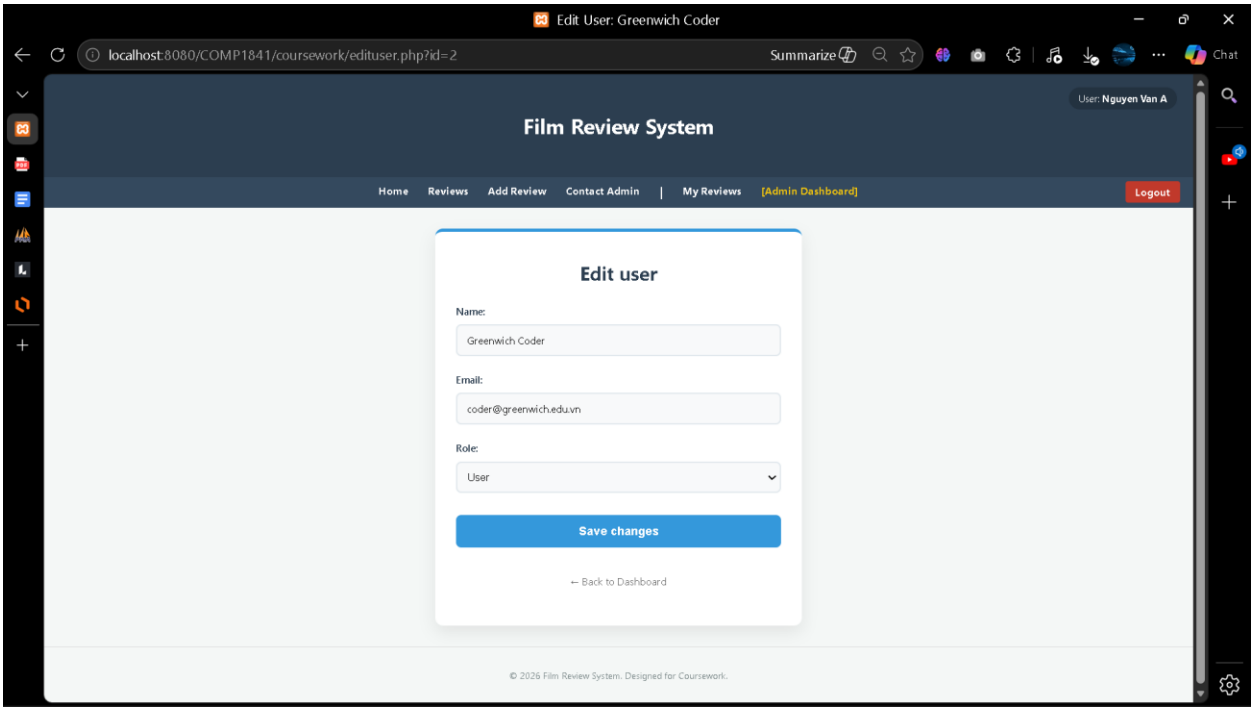
- **Create:** New users can independently register for standard accounts, establishing their unique identity within the system.



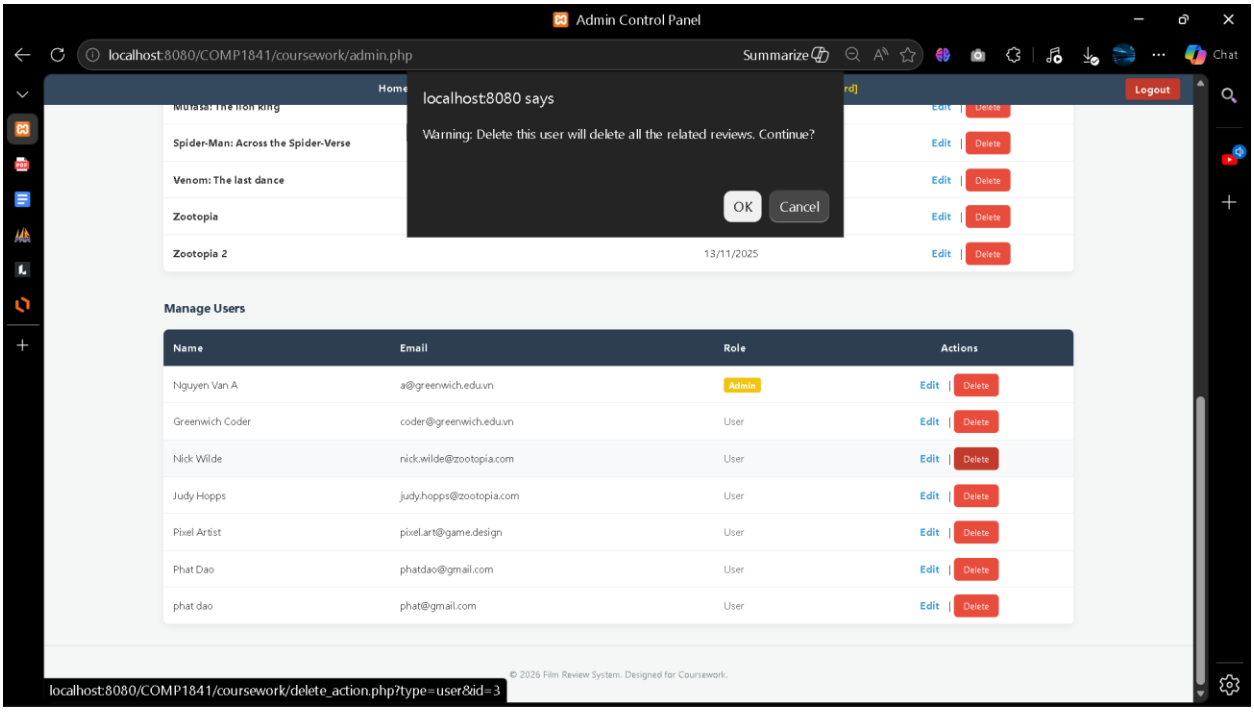
- **Read:** Administrators are provided with a comprehensive overview of all registered users within the Dashboard for auditing and monitoring purposes.



- **Update:** Administrators possess the authority to elevate user privileges (promoting them to Admin status) or edit their profile details to manage the community hierarchy.



- **Delete:** Administrators can delete user accounts to enforce community guidelines. However, logical safeguards are implemented to prevent self-deletion, mitigating accidental system lockouts.



5. Structured Testing Schedule and Input Control

5.1. Input Validation & Control

Data integrity and system security are maintained through a robust, two-tiered validation architecture:

1. **Client-side (HTML5 & JS):** This layer prioritizes User Experience (UX) by providing immediate feedback. It utilizes required attributes, specific input type constraints (e.g., type="email"), and <select> dropdowns to guide the user and prevent the submission of empty or incorrectly formatted forms before they even reach the server.
2. **Server-side (PHP/PDO):** While client-side validation enhances UX, the server-side acts as the ultimate, tamper-proof gatekeeper. It enforces trim() to remove extraneous whitespace, actively checks the database for duplicate records (e.g., verifying unique emails to prevent registration conflicts), and universally applies **PDO Prepared Statements** to sanitize all incoming data streams, neutralizing potential SQLi threats from malicious

actors who might bypass front-end controls.

5.2. Testing Schedule (Test Plan)

Test ID	Feature	Action / Input	Expected Result	Actual Result	Status
TC01	Registration	Enter an existing email address during sign-up.	System actively queries the database, rejects registration, and displays an "Email already registered" error message.	As expected	**PASS**
TC02	SQLi Defense	Enter a malicious payload: ' OR 1=1; -- as a password.	PDO engine sanitizes the input as a literal string; authentication fails safely without executing the injected logic.	As expected	**PASS**
TC03	Authorization	Non-admin user directly accesses the editfilm.php URL.	The session checker identifies insufficient privileges and immediately redirects the user back to the Home page.	As expected	**PASS**
TC04	Data Integrity	Admin deletes a User account from the dashboard.	The user account, along with all their associated reviews and nested comments, are permanently removed via SQL Cascade.	As expected	**PASS**
TC05	Login	Enter valid, registered	A secure session initializes, granting	As expected	**PASS**

		credentials.	access to restricted features, and the user redirects to the Home page.		
TC06	Login	Enter an invalid password for an existing account.	The password_verify function fails; system rejects login with a generic "Email or password is not correct!" to prevent user enumeration.	As expected	**PASS**
TC07	Comment	Post a valid text comment on a review.	The comment saves securely to the database and displays immediately in the active sidebar without breaking the layout.	As expected	**PASS**
TC08	Comment	Attempt to comment while in a logged-out state.	The input form hides defensively; the UI displays a clear message prompting the user to authenticate to join the discussion.	As expected	**PASS**
TC09	Edit Review	Author updates their review text and replaces the image.	The database updates successfully; old images are handled properly, and changes reflect on the UI immediately.	As expected	**PASS**
TC10	Delete Review	Author deletes their own review.	The review record, associated physical image files, and child comments are permanently eradicated from the system.	As expected	**PASS**

TC11	Edit Film	Admin updates a film's publish date and poster graphic.	Details update successfully, maintaining data consistency across both the administrative dashboard and public Home page.	As expected	**PASS**
TC12	Delete Film	Admin completely deletes a film entity.	The film and all historically associated reviews/comments are safely removed to prevent data fragmentation.	As expected	**PASS**
TC13	Edit User	Admin promotes a standard user to the "Admin" role.	The target user instantly gains administrative privileges, unlocking access to the previously restricted dashboard.	As expected	**PASS**
TC14	Delete User	Admin attempts to click the delete button on their own account.	The application logic blocks the action, displaying a "Cannot delete your own account" error to prevent critical administrative lockouts.	As expected	**PASS**

6. Web Accessibility (WAI)

Designed to achieve maximum inclusivity and compliance with modern standards (W3C, cited in Prichard, 2026), the system ensures that users of varying abilities can interact with the platform effectively and without frustration:

- **Visual Contrast:** High-contrast color palettes and clear typographic hierarchies are implemented to significantly aid visually impaired users, reducing cognitive load and eye strain.
- **Semantic HTML:** The strict utilization of descriptive HTML5 tags (such as <header>, <main>, and <nav>) ensures maximum compatibility with assistive technologies. This

semantic structure allows Screen Readers to accurately interpret and navigate the structural flow of the web application.

7. Discussion on Legal, Security & Business Problems

7.1. Business Problems & Crowdsourcing Strategy

A primary challenge for any new platform is the "cold start" content problem—acquiring enough initial data to attract and retain an active user base. To solve this, the system leverages a **Crowdsourcing (UGC)** strategy. While direct, detailed film addition is deliberately restricted to Administrators to maintain high data quality and consistency, regular users are empowered to indirectly input new film titles during the reviewing process. This design choice represents a calculated business trade-off: prioritizing User Experience (UX), frictionless engagement, and rapid platform growth over strict, initial data integrity. It allows Admins to step in asynchronously to moderate and complete these "No Image" placeholder films later.

7.2. Security Concerns & Mitigations

Because the platform fundamentally relies on user-submitted data, it utilizes defense-in-depth principles to mitigate a landscape of cyber threats:

- **SQL Injection (SQLi):** By treating all inputs strictly as bound parameters rather than executable code fragments, PDO Prepared Statements completely neutralize the risk of malicious query alterations.
- **Cross-Site Scripting (XSS):** To prevent attackers from injecting malicious client-side scripts that could steal session tokens or deface the site, the system employs the htmlspecialchars() function. This vital step sanitizes user-generated data before rendering it in the browser, converting potentially dangerous scripts into harmless HTML character entities.
- **Access Control & IDOR:** A stringent Role-Based Access Control (RBAC) mechanism guarantees that users can only modify or delete their *own* reviews. Meanwhile, sensitive administrative functions enforce deep is_admin session validation, actively preventing Insecure Direct Object Reference (IDOR) vulnerabilities where users might manipulate URL parameters to access restricted areas.

7.3. Legal & Privacy Compliance (GDPR Context)

Operating in a modern web environment requires strict adherence to privacy regulations. The platform aligns with core GDPR principles through **Data Minimization** (collecting only essential identifiers like Name and Email), **Secure Storage** (employing robust hashing for passwords to protect user identities in the event of a breach), and the **Right to Erasure**. By utilizing database constraints like ON DELETE CASCADE, the system guarantees that when an account is deleted,

all associated personal data, reviews, and comments are comprehensively and permanently purged from the application.

8. Conclusions and Future Recommendations

8.1. Conclusions

The project successfully delivered a highly secure, responsive, and fully functional Film Review System. Centralizing the backend logic within a dedicated DatabaseFunctions.php file ensures strong separation of concerns, resulting in excellent maintainability and scalability. The rigorous implementation of PDO effectively mitigates widespread SQLi vulnerabilities. Furthermore, the innovative CSS Flexbox and JavaScript-driven Master-Detail layout significantly elevates the User Experience (UX), providing a seamless, modern interface for community discussion.

8.2. Future Recommendations

To further elevate the platform's professional standard, the following improvements are recommended for future iterations:

1. **AJAX Integration:** Transitioning the comment submission and retrieval processes to utilize the Fetch API/AJAX. This would facilitate asynchronous interactions without disruptive page reloads, fully modernizing the application to mimic a fluid Single Page Application (SPA) feel.
2. **Server-Side Pagination:** Implementing robust server-side pagination for both the Home and Reviews pages. As the database grows, this will be critical to efficiently handle large data volumes, reducing server memory consumption and ensuring optimal page load times.
3. **Email Verification Pipeline:** Integrating a dedicated SMTP library (e.g., PHPMailer) to enforce email verification during the registration process. This would drastically enhance security, prevent the creation of spam or bot accounts, and ensure a higher quality of user engagement.

9. Professional Roles & Workplace Preparation Evaluation

Architecting and building this project provided an accurate simulation of a modern **Full-stack Web Developer** workflow. It required seamlessly integrating Front-end UI/UX principles with complex Back-end database architecture and stringent security protocols. Mastering this holistic approach—bridging the gap between user-centric design and robust server-side engineering—provides a highly valuable, practical foundation, fully preparing students for the demanding requirements of the professional technology labor market (Prichard, 2026).

10. References

1. Prichard, M. (2026) *COMP1841 Term 2 Web Programming 1 - Module Specification*. University of Greenwich.
2. Prichard, M. (2026) *The final report: Accessibility - Validation* $$
   Lecture slides (COMP1841 Web Programming 1, University of Greenwich.)
3. W3C (2024) *Web Accessibility Initiative (WAI)*. Available at: https://www.w3.org/WAI/ (Accessed: 14 April 2026).
4. PHP Framework Interop Group (2024) *PHP Data Objects (PDO)*. Available at: https://www.php.net/manual/en/book.pdo.php (Accessed: 14 April 2026).

11. AI Using Declaration

Please attach this page to the end of your assignment if you have used AI during your assignment to document how you have used it.
I have used AI while undertaking my assignment in the following ways:

- To develop research questions about the topic – NO
- To outline the topic – NO
- To explain concepts – YES
- To support my use of language – YES

12. Github link

PhatDaoo/coursework